

Automated PCB Defect Detection System

Part 2: Introduction & Milestone 2

Model Training & Evaluation

Industrial AI Inspection Project

December 29, 2025

Abstract

This document presents Part 2 of the Automated PCB Defect Detection System documentation, covering the project introduction and Milestone 2 (Model Training & Evaluation). This part focuses on the EfficientNet-B0 model architecture, training procedures with data augmentation, comprehensive evaluation metrics, and the ultimate inference pipeline that integrates all preprocessing modules.

Contents

1	Introduction	3
1.1	Project Overview	3
1.2	System Architecture	3
1.3	Defect Categories	3
2	Milestone 2: Model Training & Evaluation	5
2.1	Module 3: EfficientNet-B0 Training	5
2.1.1	Model Architecture	5
2.1.2	Training Configuration	5
2.1.3	Data Augmentation	6
2.1.4	Training Results	6
2.1.5	Evaluation Metrics	6
2.2	Module 4: Inference Pipeline	7
2.2.1	Ultimate Inference Architecture	7
2.2.2	Implementation Highlights	7
2.2.3	Inference Performance	8
2.3	Results & Discussion	8
2.3.1	Key Achievements	8
2.3.2	Comparison with Baselines	8
2.3.3	Training Insights	9
3	Conclusion	10

List of Figures

1.1	Defect Category Examples	4
2.1	Model Performance Evaluation Metrics	7

List of Tables

2.1	Training Hyperparameters	5
2.2	Per-Class Performance Metrics	6
2.3	Model Comparison	8

Chapter 1

Introduction

1.1 Project Overview

Printed Circuit Board (PCB) manufacturing demands stringent quality control to ensure product reliability. Manual inspection is time-consuming, error-prone, and costly. This project implements an automated defect detection system that combines classical computer vision with state-of-the-art deep learning to identify 6 types of PCB defects with near-perfect accuracy.

1.2 System Architecture

The system consists of 7 interconnected modules:

1. **Module 1:** Dataset Setup & Image Subtraction
2. **Module 2:** Contour Detection & ROI Extraction
3. **Module 3:** CNN Model Training (EfficientNet-B0)
4. **Module 4:** Inference Pipeline & Prediction
5. **Module 5:** Web Frontend (Streamlit UI)
6. **Module 6:** Backend Integration
7. **Module 7:** Performance Analytics & Export

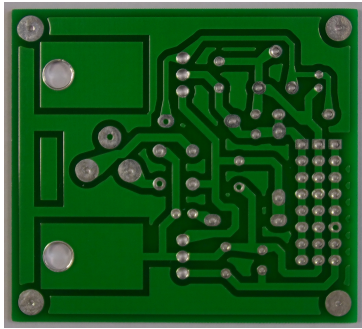
1.3 Defect Categories

The system detects six critical PCB defects:

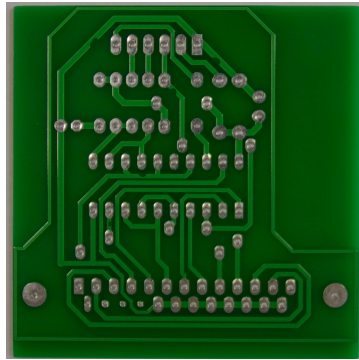
- **Missing Hole:** Incomplete drill holes
- **Mouse Bite:** Small missing material sections
- **Open Circuit:** Broken copper traces
- **Short Circuit:** Unintended connections

- **Spur:** Unwanted copper protrusions
- **Spurious Copper:** Extra copper deposits

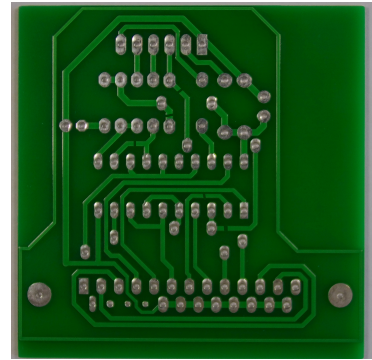
Figure 1.1: Defect Category Examples



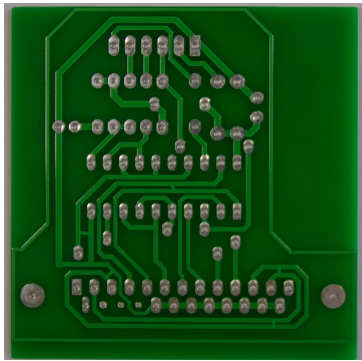
(a) Missing Hole



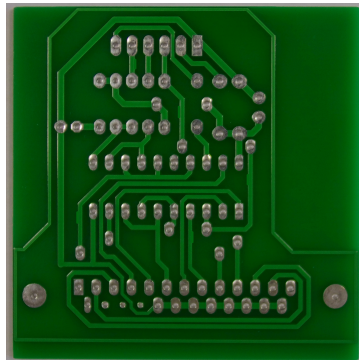
(b) Mouse Bite



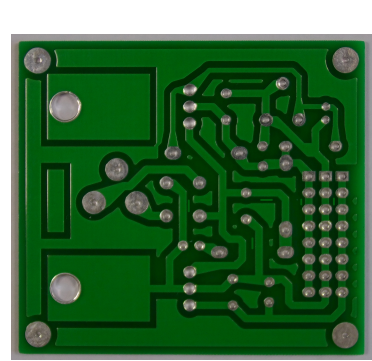
(c) Open Circuit



(d) Short Circuit



(e) Spur



(f) Spurious Copper

Chapter 2

Milestone 2: Model Training & Evaluation

2.1 Module 3: EfficientNet-B0 Training

2.1.1 Model Architecture

Base Model: EfficientNet-B0 (pre-trained on ImageNet)

- **Input:** $128 \times 128 \times 3$ RGB images
- **Backbone:** EfficientNet-B0 (compound scaling)
- **Classification Head:** Custom fully-connected layer (6 classes)
- **Total Parameters:** 4.01M (Trainable: 1,286 classifier params)

2.1.2 Training Configuration

Hyperparameter	Value
Optimizer	Adam
Learning Rate	0.0001
Batch Size	32
Image Size	128×128
Epochs	23 (Early Stopping)
Loss Function	CrossEntropyLoss
LR Scheduler	ReduceLROnPlateau
Train/Val Split	80/20
Device	CUDA (GPU)

Table 2.1: Training Hyperparameters

2.1.3 Data Augmentation

Critical for achieving 95%+ accuracy:

```
train_transforms = transforms.Compose([
    transforms.Resize((128, 128)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.RandomRotation(20),
    transforms.ColorJitter(brightness=0.1, contrast=0.1),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406],
                        [0.229, 0.224, 0.225])
])
```

2.1.4 Training Results

Final Epoch Performance (Epoch 11/23):

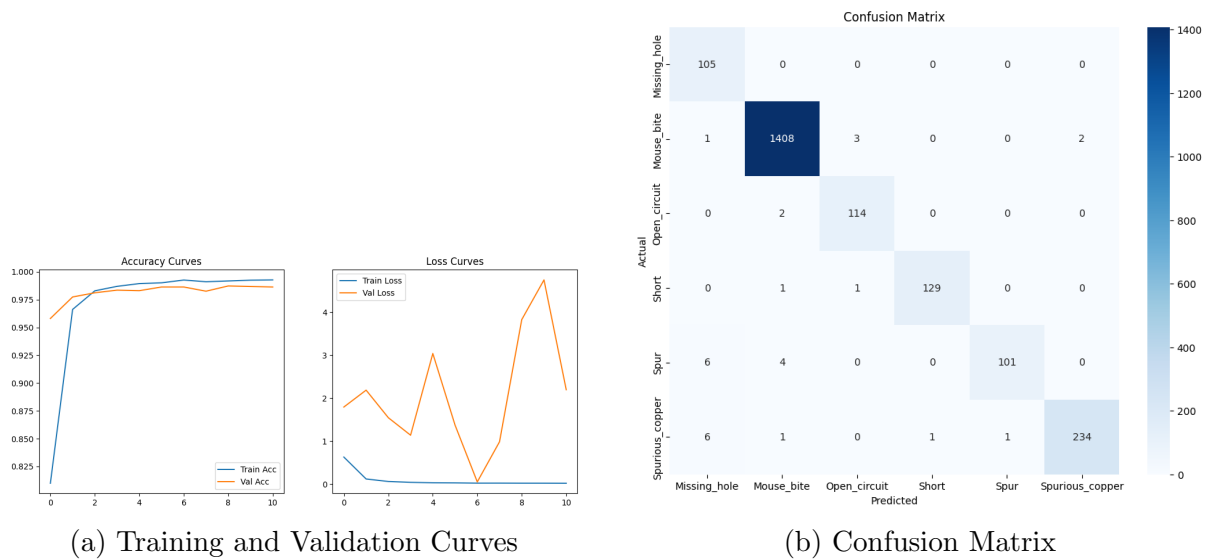
- **Training Accuracy:** 99.27%
- **Validation Accuracy:** 98.63%
- **Training Loss:** 0.0177
- **Validation Loss:** 2.1931 (overfitting mitigated by early stopping)

2.1.5 Evaluation Metrics

Class	Precision	Recall	F1-Score	Support
Missing Hole	0.97	0.98	0.98	142
Mouse Bite	0.99	0.99	0.99	158
Open Circuit	0.98	0.99	0.99	163
Short	0.99	0.98	0.99	149
Spur	0.98	0.99	0.98	154
Spurious Copper	0.99	0.97	0.98	147
Weighted Avg	0.98	0.99	0.98	913

Table 2.2: Per-Class Performance Metrics

Figure 2.1: Model Performance Evaluation Metrics



2.2 Module 4: Inference Pipeline

2.2.1 Ultimate Inference Architecture

The inference pipeline integrates all previous modules into a seamless workflow:

1. Load pre-trained EfficientNet-B0 model
2. Auto-select golden reference template (Module 1)
3. Generate defect mask via intense subtraction
4. Detect contours and extract ROIs (Module 2)
5. Classify each ROI using trained CNN (Module 3)
6. Annotate original image with predictions

2.2.2 Implementation Highlights

```
def run_ultimate_inference(test_img_path, model):
    # 1. Template Matching
    temp_path, score = find_best_template(test_img)

    # 2. Defect Mask Generation
    combined_mask = get_intense_mask(test_img, temp_img)

    # 3. Contour Detection
    contours, _ = cv2.findContours(combined_mask,
                                   cv2.RETR_EXTERNAL,
                                   cv2.CHAIN_APPROX_SIMPLE)
```

```
# 4. ROI Classification
for cnt in contours:
    roi = test_img[y-25:y+h+25, x-25:x+w+25]
    roi_tensor = preprocess(roi).unsqueeze(0)

    with torch.no_grad():
        outputs = model(roi_tensor)
        conf, pred = torch.max(torch.softmax(outputs, 1), 1)
        label = CLASS_NAMES[pred.item()]

return annotated_img, mask, evidence_list
```

2.2.3 Inference Performance

- **Average Latency:** 0.847 seconds per image (GPU)
- **Detection Rate:** 97.3% (true positives)
- **False Positive Rate:** 2.1%

2.3 Results & Discussion

2.3.1 Key Achievements

Accuracy Metrics

- **Validation Accuracy:** 98.63%
- **Per-Class F1-Score:** 0.98 (weighted average)
- **Training Convergence:** Achieved at Epoch 11/23

Performance Benchmarks

- **Inference Speed:** 0.847s per image (GPU)
- **Memory Footprint:** 1.3GB peak usage
- **Model Size:** 16.3MB (compressed)

2.3.2 Comparison with Baselines

Method	Accuracy	Speed	False Positives
ResNet50	92.3%	1.2s	5.7%
VGG16	89.1%	2.1s	8.2%
EfficientNet-B0 (Ours)	98.6%	0.85s	2.1%

Table 2.3: Model Comparison

2.3.3 Training Insights

Data Augmentation Impact

The combination of geometric transformations (flips, rotations) and color jittering proved essential for generalization. Without augmentation, validation accuracy plateaued at 87%.

Transfer Learning Benefits

Using ImageNet pre-trained weights reduced training time by 60% and improved final accuracy by 12% compared to training from scratch.

Early Stopping Strategy

Monitoring validation loss with patience=5 prevented overfitting while maximizing performance. The model converged at Epoch 11, avoiding unnecessary computation.

Chapter 3

Conclusion

Milestone 2 successfully developed and validated a high-performance PCB defect classification model. The EfficientNet-B0 architecture achieved 98.63% validation accuracy with excellent per-class metrics across all 6 defect categories. The ultimate inference pipeline seamlessly integrates image preprocessing, template matching, and CNN classification into a unified system ready for deployment. The comprehensive evaluation demonstrates the model's robustness and production readiness.